

MPI Virtual Topologies

Ernesto Sanchez

Virtual topologies

Once a communicator is created, the physical topology may not fit the communication requirements needed by the logical topology.

- Each process or processor in the group is assigned a rank between 0 and $N-1$.
- In many parallel applications a linear ranking of processes does not adequately reflect the logical communication pattern of the processes (which is usually determined by the underlying problem geometry and the numerical algorithm used).

Virtual topologies

A *virtual topology* allows the MPI library to optimize communications by exploiting the knowledge about the physical topology.

A **physical topology** represents the actual connections between the cores, chips, and nodes in the real hardware.

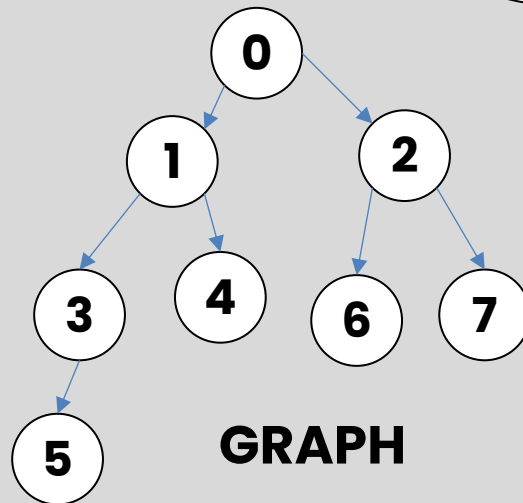
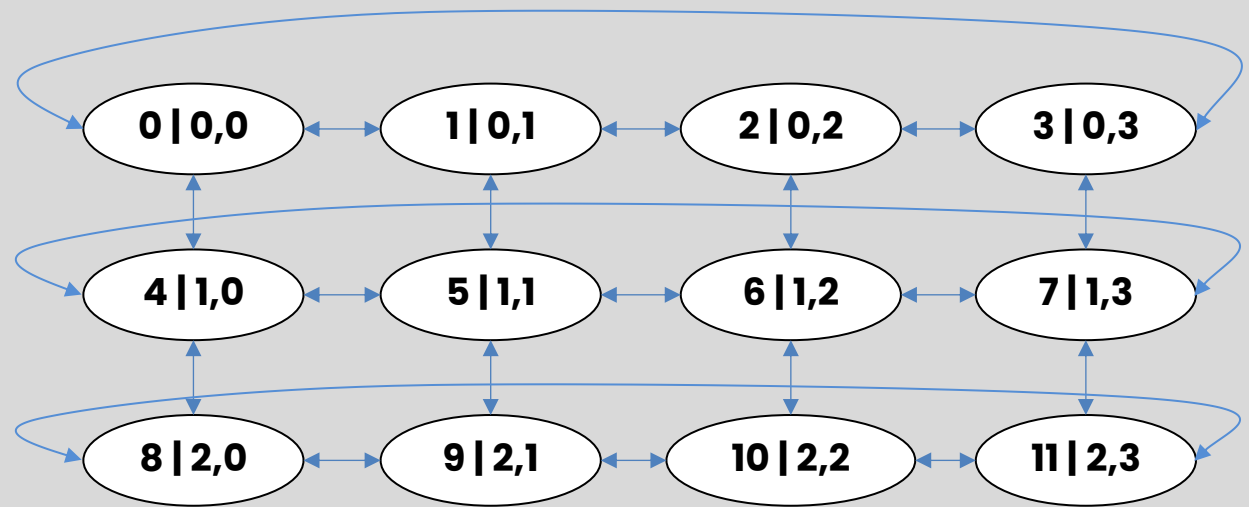
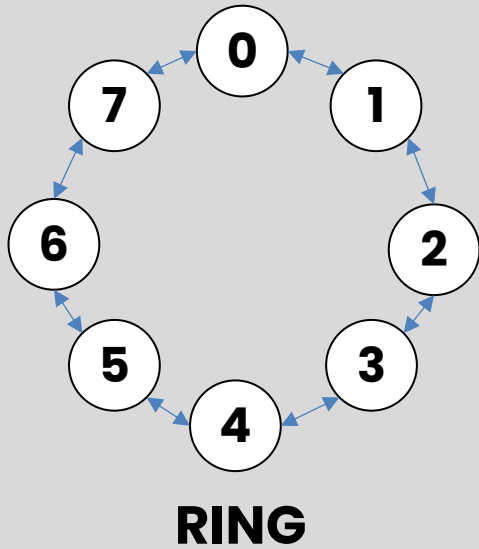
A **virtual topology** represents the way that MPI processes communicate:

- Nearest neighbor exchange in a mesh
- In an all-to-all exchange, a process is allowed to send and receive a fixed amount of data from every other process

The description of the virtual topology depends only on the application, and it is machine-independent.

MPI supports **Cartesian** and **Graph** topologies.

Virtual topologies - examples



MPI Virtual Topologies

MPI provides routines to create new communicators that order the processes ranks, looking at better match the physical topology of the real hardware.

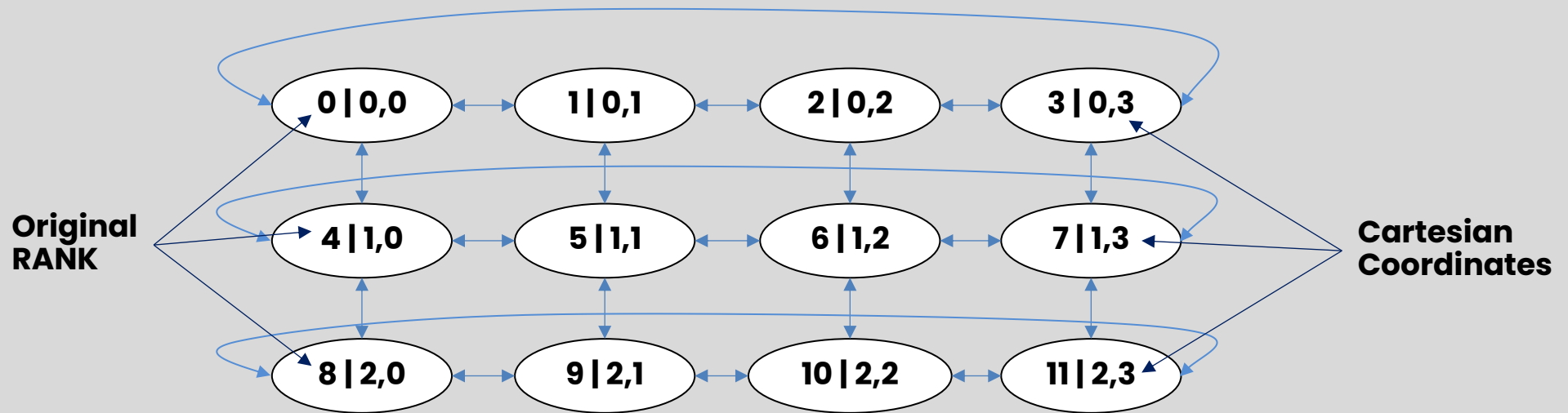
Special functions facilitate the creation and access to the defined virtual topology:

- The virtual topology is a property of a communicator
- Topology routines create a new communicator with properties of the specified virtual topology.

Cartesian topology

A grid or regular network of processes can be described with a cartesian topology:

- each process can be identified by cartesian coordinates (x, y)
- periodicity can be selected for each direction
- communications are performed along grid dimensions only.



MPI Cartesian Topology

MPI provides functions for:

- Creating a new cartesian topology
- Determining a good organization of the topology according the system characteristics
- Getting information about the topology.

Create an MPI Cartesian Topology

`MPI_Cart_create()`

Creates a new communicator `comm_cart` from `comm_old`, that represents a *ndim* dimensional mesh with sizes *dims*[]. The mesh is periodic in a coordinate *i* if `periods[i]` is true. The ranks in the new communicator are reordered (to better match the physical topology) if `reorder` is true.

```
int MPI_Cart_create(  
    MPI_Comm comm_old,    // Input communicator  
    int ndims,            // # of dimensions of Cartesian grid (integer).  
    const int dims[],     // Integer array of size ndims specifying the number of  
                          // processes in each dimension.  
    const int periods[],  // Logical array of size ndims specifying whether the grid is  
                          // periodic (true) or not (false) in each dimension.  
    int reorder,          // Ranking may be reordered (true) or not (false) (logical).  
    MPI_Comm *comm_cart) // Communicator with new Cartesian topology (handle).
```

Example MPI Cartesian Topology

```
int dims[2]={3,4}, periods[2]={1,0};
MPI_Cart_create(
    MPI_COMM_WORLD, // Input communicator
    2,              // # of dimensions of Cartesian grid (integer).
    dims[],         // Integer array of size ndims specif. the number of processes in each dim.
    periods[],      // Logical array of size ndims specifying whether the grid is periodic.
    0,             // Reorder the ranking (FALSE)
    &comm_cart)     // Communicator with new Cartesian topology (handle).
```

Here, it is assumed that we count with a 12 processes initial communicator.



Finding an optimal Cartesian Topology

`MPI_Dims_create()`

Populates an array *dims*[*ndims*] providing a balanced distribution of processes in every dimension. The number of distributed processes is *nnodes*.

Any value of *dims*[*i*] that is 0 on input will be replaced; values that are > 0 will not be changed.

The resulting *dims*[] array is suitable for use as input to the routine `MPI_Cart_create`.

```
int MPI_Dims_create(  
    int nnodes,    // Number of nodes in a grid (integer).  
    int ndims,     // # of dimensions of Cartesian grid (integer).  
    int dims[])    // Integer array of size ndims specifying the number  
                  // of processes in each dimension
```

Finding an optimal Cartesian Topology

```
int MPI_Dims_create(
    int nnodes,    // Number of nodes in a grid (integer).
    int ndims,     // # of dimensions of Cartesian grid (integer).
    int dims[])    // Integer array of size ndims specifying the
                  // number of processes in each dimension.
```

dims before call	function call	dims on return
{0,0}	MPI_Dims_create(6, 2, dims)	{3,2}
{0,0}	MPI_Dims_create(7, 2, dims)	{7,1}
{0,3,0}	MPI_Dims_create(6, 3, dims)	{2,3,1}
{0,3,0}	MPI_Dims_create(7, 3, dims)	erroneous call

0|0,0

1|0,1

2|1,0

3|1,1

4|2,0

5|2,1

Find my coordinates in the communicator

`MPI_Cart_coords()`

Determines process coordinates in a Cartesian topology given a rank in the group.

```
int MPI_Cart_coords(  
    MPI_Comm comm, // communicator with cartesian structure  
    int rank,      // rank of a process within group of comm  
    int maxdims,   // length of vector coords  
    int *coords )  // output integer array (of ndims)
```

Find my rank given the coordinates in the communicator

`MPI_Cart_rank()`

Determines process rank in communicator given Cartesian location.

```
int MPI_Cart_rank(  
    MPI_Comm comm_cart, // Communicator with Cartesian structure  
    int coords[],       // Integer array (of size ndims) having the  
                        // process coordinates  
    int *rank)          // Rank of specified process (integer).
```

Other MPI functions

`int MPI_Cart_sub()`

Partitions a communicator into subgroups which form lower-dimensional cartesian subgrids.

`int MPI_Cart_shift()`

Returns the shifted source and destination ranks, given a shift direction and amount from the current process.

`int MPI_Group_union()`

Creates a group by combining two groups.

`int MPI_Group_compare()`

Compares group members and group order.

MPI Graph Topology

MPI provides functions for:

- Creating a new graph topology
- Graph vertices represent MPI processes (usually one per process)
- Graph edges indicate important connections
- Edge weights provide more information.

Create an MPI Graph Topology

`MPI_Graph_create()`

constructs a communicator from the graph topology information passed. When using this approach, every MPI process must provide the entirety of graph virtual topology information, which may hinder scalability.

```
int MPI_Graph_create(  
    MPI_Comm old_comm, //Communicator containing the processes to use  
    int number_of_nodes,  
    const int index[], //Array containing the accumulating number of neighbours  
    const int edges[], //The list of edges in the graph  
    int reorder,  
    MPI_Comm* new_comm);
```